Лабораторна робота № 4

Частина 2

Застосування нейронних мереж у робототехнічних системах для детекції об’єктів на зображеннях (YOLO) та визначення стану системи за вимірами вбудованих сенсорів (LSTM, GRU)

Мета роботи

Ознайомитися із застосуванням глибоких нейронних мереж у робототехніці та пройти повний цикл побудови двох типових компонентів робототехнічної системи: модуля комп’ютерного зору для детекції об’єктів і модуля аналізу послідовностей сенсорних вимірювань. Навчитися готувати дані, донавчати YOLO на власному наборі, отримувати передбачення на основі навченої моделі, інтерпретувати вихідні структури моделі, обчислювати метрики детекції, а також будувати й оцінювати LSTM/GRU-класифікатор для послідовностей даних сенсорів робота.

Теоретичний матеріал

У робототехнічних системах нейронні мережі часто використовуються для обробки даних, отриманих від камер і сенсорів. Залежно від типу вхідних даних застосовуються різні архітектури нейронних мереж.

У цій лабораторній роботі буде розглянуто два типові завдання:

- детекція об’єктів на зображеннях за допомогою моделей сімейства YOLO;
- аналіз часових послідовностей сенсорних даних за допомогою рекурентних нейронних мереж LSTM та GRU.

YOLO дозволяє визначати, які об’єкти присутні на зображенні та де вони розташовані. У робототехніці такі моделі можуть використовуватися, наприклад, для виявлення людей, транспортних засобів, дорожніх знаків, перешкод або інших об’єктів у полі зору камери.

LSTM та GRU призначені для роботи з послідовностями даних, де важливим є не лише поточне значення, а й попередні вимірювання. У робототехніці такими даними можуть бути послідовності показників датчиків відстані, інерційних сенсорів, енкодерів або інших сенсорів. Аналіз часової послідовності дозволяє моделі враховувати зміну стану системи в часі.

Детекція об’єктів за допомогою YOLO

На відміну від класифікатора зображень, що отримує на вхід вже вирізаний об’єкт і визначає його клас, детектор повинен одночасно відповідати на два питання:

1. Що знаходиться на зображенні?
2. Де саме знаходиться кожен об’єкт?

Результатом роботи детектора зазвичай є: обмежувальні рамки (bounding boxes), класи об’єктів, а також показники впевненості в наявності об’єктів в рамках (confidence).

YOLO (You Only Look Once) - сімейство одноетапних детекторів, у яких одна нейронна мережа обробляє все зображення та безпосередньо формує передбачення щодо наявних об’єктів. Оригінальна стаття, де представлено архітектуру YOLO, сформулювала детекцію як єдину задачу регресії координат обмежувальних рамок та ймовірностей класів. Сучасні версії YOLO суттєво відрізняються від першої архітектури, але зберігають основну ідею швидкого наскрізного детектора.

Саме одноетапна архітектура YOLO, де знаходження об’єктів і їх класифікація відбувається одночасно, надає її перевагу у швидкодії над іншими моделями, такими, як Faster R-CNN, що виконують ці підзадачі. Швидкодія YOLO, а також її малий розмір роблять дану модель придатною для використання у робототехнічних системах, де детекція найчастіше має виконуватись в реальному часі.

Архітектуру Ultralytics YOLO26 зручно умовно поділити на три частини:

- **backbone -** основна мережа виділення ознак;
- **neck -** проміжна частина, що об’єднує ознаки різних масштабів;
- **detection head -** вихідна частина детектора, що формує координати рамок і передбачення класів;

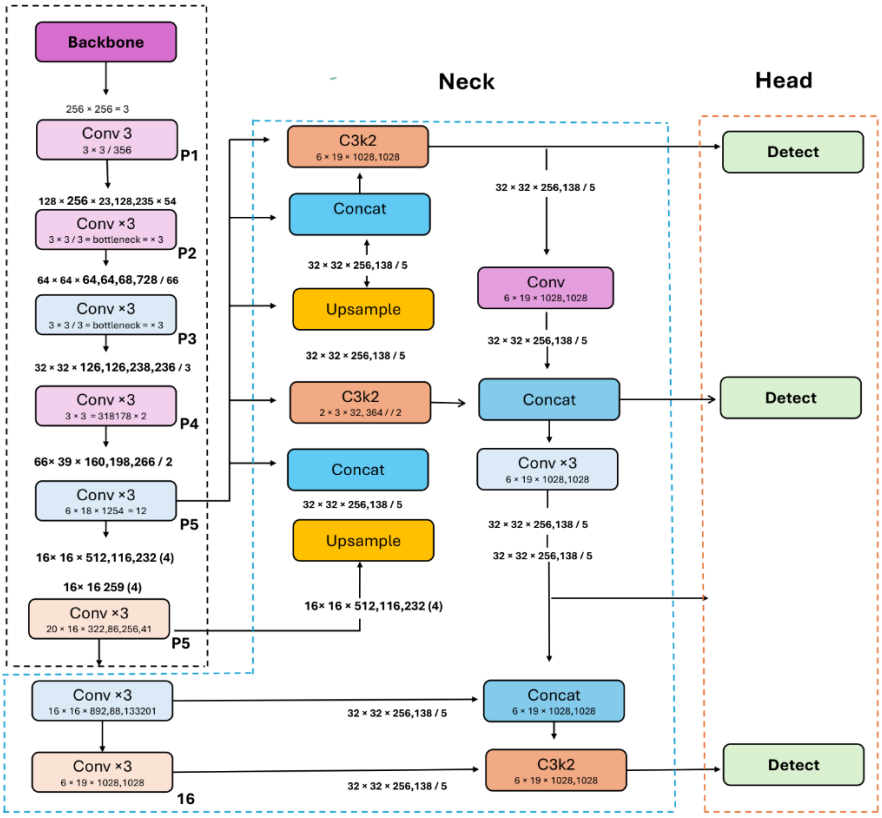


Рис. 1. Архітектура YOLO26

Формат набору даних для задачі детекції з допомогою YOLO

Типовий набір даних для YOLO має наступну структуру:

```
dataset/
  data.yaml
  train/
    images/
    labels/
  val/
    images/
    labels/
  test/
    images/
    labels/
```

- **data.yaml** – визначає навчальну, валідаційну і тестову вибірки та класи
- **train, val, test** – містять тренувальні, валідаційні і тестові дані
- **images** – містить файли зображення відповідної вибірки з набору даних
- **labels** – містить txt файли з анотаціями (класи та обмежувальні рамки об'єктів на відповідних зображеннях)

```yaml
train: ../train/images
val: ../valid/images
test: ../test/images

nc: 8
names: ['construction', 'flashing 40', 'green', 'red', 'speed', 'stop', 'yellow', 'yield']
```

Рис 2. Приклад data.yaml

В файлах з анотаціями один рядок описує один об'єкт:

```
class_id x_center y_center width height
```

- class_id – номер класу;
- x_center, y_center – координати центра рамки;
- width, height – ширина та висота рамки;

Координати нормалізовані до діапазону [0, 1] відносно ширини й висоти зображення.

Тренування YOLO в Python

YOLO26 для детекції об'єктів представлена декількаами варіантами різного розміру, що дозволяє підібрати найкращу модель на основі характеристик робототехнічної системи та

вимог до швидкодії і точності.

Model	size (pixels)	mAP^{val} 50-95	mAP^{val} 50-95(e2e)	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO26n	640	40.9	40.1	**38.9 ± 0.7**	**1.7 ± 0.0**	**2.4**	**5.5**
YOLO26s	640	48.6	47.8	87.2 ± 0.9	2.5 ± 0.0	9.5	20.9
YOLO26m	640	53.1	52.5	220.0 ± 1.4	4.7 ± 0.1	20.4	68.4
YOLO26l	640	55.0	54.4	286.2 ± 2.0	6.2 ± 0.2	24.8	86.8
YOLO26x	640	**57.5**	**56.9**	525.8 ± 4.0	11.8 ± 0.2	55.7	194.4

Рис 3. Варіанти моделі YOLO26

Для використання YOLO в Python попередньо треба встановити бібліотеку ultralytics:

```
$ pip install ultralytics
```

Приклад завантаження моделі в пам’ять:

```
from ultralytics import YOLO
      model = YOLO("yolo26n.pt")  # Завантаження претренованої моделі
```

Також бібліотека дозволяє завантажити модель без попереднього претренування, для цього в коді вище потрібно замінити формат з pt на yaml.

Для тренування YOLO використовується метод train, що можна застосувати так:

```
train_results = model.train(
    data="TrafficSignsDataset/data.yaml",
    epochs=50,
    patience=20,
    imgsz=640,
    batch=16,
    project="TrafficSigns",
    name="TestExperiment",
    plots=True,
    save=True,
    exist_ok=False,
    verbose=True,
    device="cuda",
    mosaic=0.4,
    scale=0.4,
    fliplr=0.4,
    translate=0.1,
    close_mosaic=10,
    seed=0
)
```

Після навчання у каталозі запуску зберігаються (всередині папки runs):

- best.pt – контрольна точка (checkpoint) із найкращою валідаційною якістю;
- last.pt - ваги після останньої епохи;

- results.csv - числові результати навчання;
- results.png - графіки функцій втрат і метрик;
- приклади навчальних пакетів;
- матриця помилок;
- криві метрик;

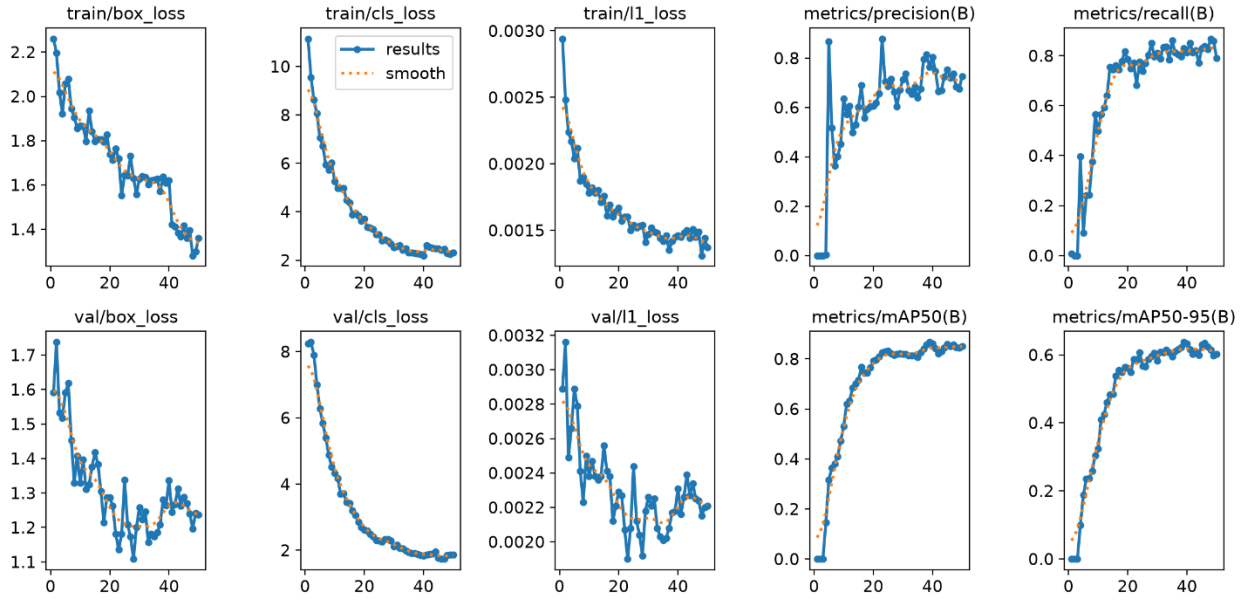


Рис. 4. Приклад графіків функцій втрат і метрик автоматично створених під час тренування YOLO

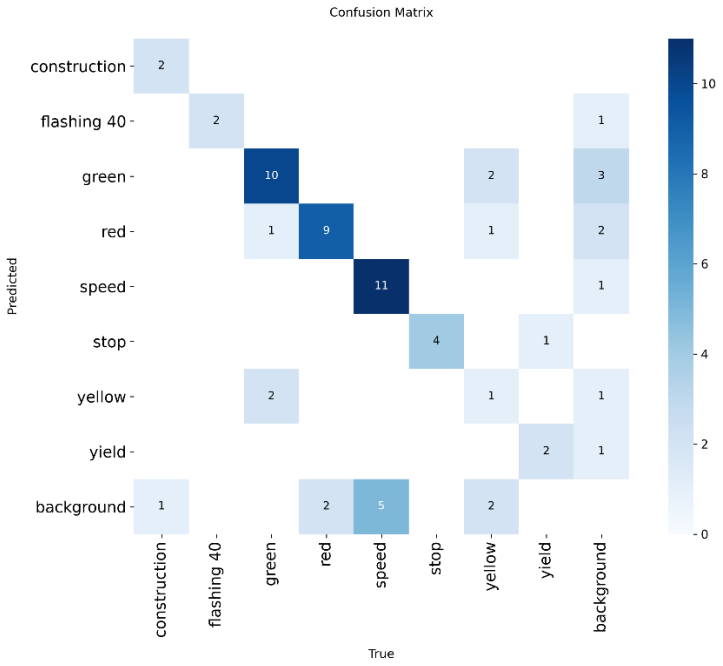


Рис. 5. Приклад матриці невідповідностей (confusion matrix) автоматично створеної під час тренування YOLO

В наступних таблицях можна побачити опис найважливіших параметрів методу `train`, більш детальну інформацію можна знайти в документації Ultralytics.

Таблиця 1. Основні параметри методу `train`

Параметр	**Приклад / тип**	**Призначення**
data	Шлях до data.yaml	Опис train/val/test та класів
epochs	Кількість епох	Максимальна кількість повних проходів по навчальній вибірці
imgsz	Розмір вхідного зображення	Розмір до якого трансформується зображення перед подачею на вхід в нейронну мережу
batch	Розмір пакета	Кількість оброблених зображень за один оптимізаційний крок
device	“cpu” / “cuda” / індекс GPU	Пристрій для навчання (використання cuda потребує встановлення PyTorch з її підтримкою)
patience	Кількість епох без покращень	Зупиняє навчання, якщо метрики не покращуються впродовж даної кількості епох
optimizer	“auto” або конкретний оптимізатор	Спосіб оновлення ваг
lr0	Початковий learning rate	Величина кроку оптимізації
weight_decay	L2-регуляризація	Допомагає контролювати перенавчання
pretrained	True/False або ваги	Вказує, чи використовувати претреновані ваги
freeze	Номери/кількість шарів	Заморожує частину ваг
cache	True/“ram”/“disk”	Кешування набору даних для швидкого тренування
workers	Кількість потоків	Паралельне завантаження даних
seed	random seed	Відтворюванність експериментів
project, name	Каталог і назва запуску	Організація результатів експериментів. Для порівняльних експериментів кожному запуску варто задавати окреме значення name.
plots	True/False	Збереження стандартних графіків і діагностики

Таблиця 2. Параметри аугментації методу `train`

Параметр	Що робить	Призначення
mosaic	Об'єднує кілька зображень в одне	Збільшує різноманітність масштабів, позицій і контекстів
close_mosaic	Вимикає mosaic протягом останніх N епох	Дозволяє завершити навчання на більш природних зображеннях
fliplr / flipud	Воризонтальне / вертикальне віддзеркалення	Корисно, якщо орієнтація об'єкта не критична
scale	Змінює масштаб сцени та об'єктів	Імітує різну відстань до камери
translate	Зміщує зображення	Імітує різні положення об'єкта в кадрі
degrees	Повертає зображення на заданий кут	Корисно для камер з різною орієнтацією
hsv_h / hsv_s / hsv_v	Зміна відтінку/насиченості/яскравості	Імітує освітлення й різні камери
mixup	Накладає два зображення	Може покращити регуляризацію
copy_paste	Переносить об'єкти або маски між зображеннями	Збільшує кількість і різноманітність об'єктів

Також ultralytics дозволяє відстежувати тренування YOLO в TensorBoard, для цього потрібно виконати наступні команди в терміналі:

```
$yolo settings tensorboard=True
$tensorboard --logdir runs
```

Тут runs це папка, яку створює YOLO при тренуванні в папці, де виконується Python програма.

Отримання передбачень YOLO в Python

Приклад використання вже навченої моделі для отримання передбачень на основі шляху до зображення:

```
from pathlib import Path

experiment_dir = Path(train_results.save_dir)
best_model_path = experiment_dir / "weights" / "best.pt"
best_model = YOLO(str(best_model_path))
```

```
results = best_model.predict("image.jpg", imgsz=640, conf=0.25)
```

Таблиця 3. Основні параметри методу `predict`

Параметр	**Призначення**
source	Шлях до папки з зображеннями, файлу зображення, чи зображення завантаженого в пам'ять (наприклад, через opencv)
conf	Ігнорує передбачення з меншим значенням впевненості
imgsz	Зображення, що передається на вхід
device	Дозволяє виконувати обчислення на процесорі, чи відеокарті
max_det	Максимальна кількість детекцій на одному зображенні
classes	Дозволяє отримувати передбачення тільки для вказаного списку класів

Метод `predict` повертає список об'єктів `Results` по одному для кожного вхідного зображення або кадру.

Таблиця 4. Основні поля об'єктів, що повертає `predict`

Поле	**Призначення**
result.boxes	Контейнер усіх знайдених обмежувальних рамок
boxes.xyxy	Координати двох вершин обмежувальної рамки для кожного передбачення
boxes.conf	Рівень впевеності для кожного передбачення
boxes.cls	Ідентифікатор класу для кожного передбачення
result.names	Відповідність ідентифікаторів класів їх назвам

Також `Results` дозволяє показувати зображення з спрогнозованими обмежувальними рамками і класами:

```
import matplotlib.pyplot as plt
for result in results:
    plt.figure(figsize=(10, 6))
    plt.imshow(result.plot()[..., ::-1])
    plt.axis("off")
    plt.tight_layout()
    plt.show()
```

Метрики моделі для детекції об'єктів

IoU (Intersection over Union) – відношення площі перетину передбаченої обмежувальної рамки і еталонної рамки до площі їх об'єднання.

Зазвичай в задачі детекції об'єктів передбачення вважать правильним, якщо:

1. Передбачений клас правильний.
2. IoU перевищує заданий поріг, наприклад 0.5.

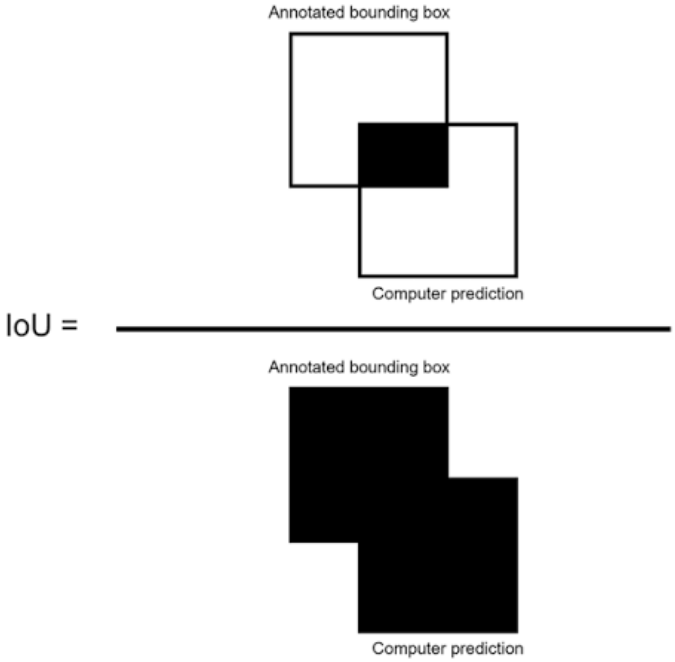


Рис. 6. Геометричний зміст IoU

Precision – точність позитивних передбачень. Показує, яка частка всіх знайдених моделлю об’єктів була правильною:

$$Precision = \frac{TP}{TP + FP}$$

Високий Precision важливий, коли хибні спрацьовування дорогі. Наприклад, якщо складський робот аварійно зупиняється після детекції людини, велика кількість FP може суттєво знизити продуктивність системи.

Recall – повнота. Показує, яку частку реальних об’єктів модель змогла знайти:

$$Recall = \frac{TP}{TP + FN}$$

Високий Recall особливо важливий у системах, пов’язаних із безпекою, де пропуск реального об’єкта може бути небезпечнішим за зайву детекцію.

F1-score – середнє гармонійне між Precision і Recall:

$$F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}$$

F1 зручно використовувати для вибору порогу confidence, коли необхідно збалансувати FP та FN.

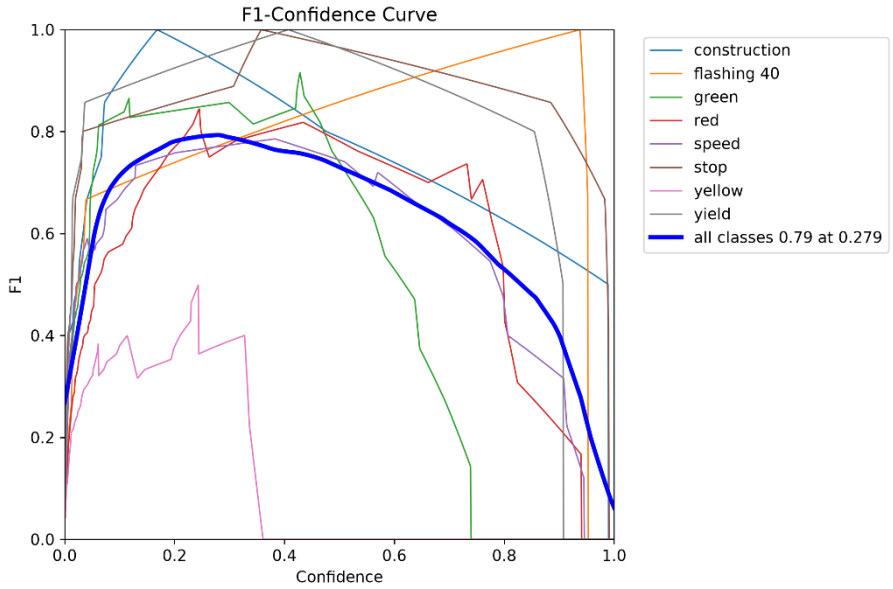


Рис. 7. Приклад залежності F1-score від порогу confidence автоматично згенерований при тренуванні YOLO

PR-крива (Precision-Recall curve) утворюється шляхом зміни порога впевненості та обчислення Precision і Recall для кожного значення.

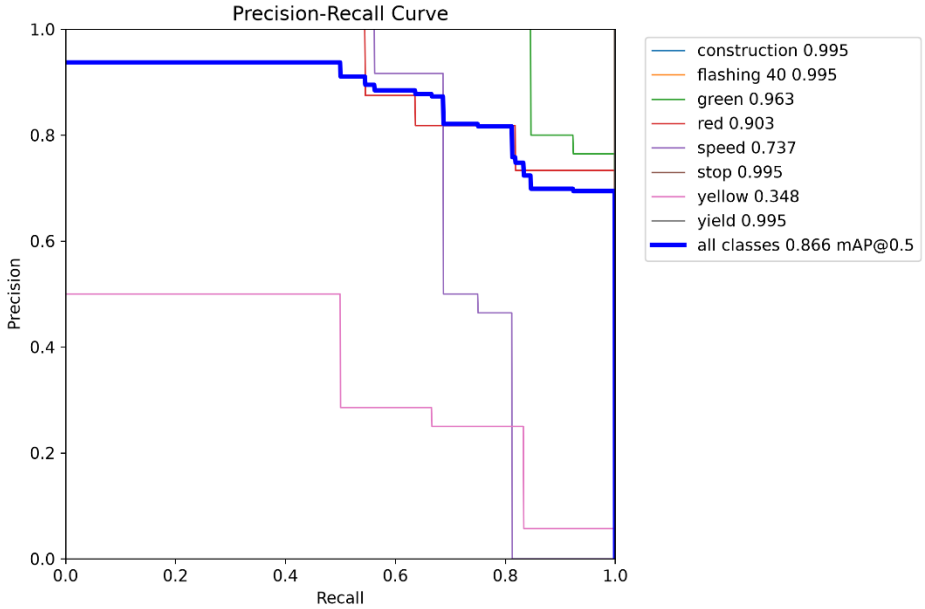


Рис. 8. Приклад PR-кривої автоматично згенерованої при тренуванні YOLO

AP (Average Precision) – площа під інтерпольованою PR-кривою для одного класу при заданому порозі IoU.

mAP (mean Average Precision) – AP, усереднене за класами.

mAP50 – AP обчислюється при $IoU = 0.5$, після чого результати усереднюються за класами.

mAP50–95 – AP обчислюється при порогах $0.5, 0.55, 0.6, \ldots, 0.95$, після чого значення усереднюються за порогами та класами. Ця метрика значно суворіше оцінює точність обмежувальних рамок.

Іншими корисними показниками можуть бути: матриця помилок (confusion matrix), час одного передбачення, розмір моделі, значення функції втрат на навчальній і валідаційній вибірці, використання пам'яті, а також час навчання.

Валідація YOLO в Python

Валідацію моделі на тестовому наборі даних можна виконати наступним чином:

```
metrics = best_model.val(
    data="TrafficSignsDataset/data.yaml",
    split="test",
    imgsz=640,
    batch=16,
    plots=True,
)
print("Precision:", metrics.box.mp)
print("Recall:", metrics.box.mr)
print("mAP@50:", metrics.box.map50)
print("mAP@50-95:", metrics.box.map)
print("Per-class mAP@50-95:", metrics.box.maps)
```

При значенні аргумента `plots=True`, під час валідації автоматично будуть створенні візуалізації різних метрик, так само, як і під час тренування моделі. Їх можна знайти в відповідній папці всередині папки runs.

LSTM/GRU для аналізу сенсорних послідовностей

При обробці вимірів сенсорів робототехнічної системи, їх значень в конкретний момент часу може бути недостатньо для класифікації поточного стану системи. Наприклад, однакові відстані до стін можуть спостерігатися як під час прямолінійного руху, так і перед початком повороту. Цю проблему можна вирішити працюючи з послідовністю вимірювань, саме для роботи з ними використовують RNN, LSTM та GRU.

RNN (Recurrent Neural Network) має внутрішній прихований стан h_t, який обраховується на основі попереднього стану h_{t-1}, та поточного входу мережі x_t:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Під час зворотного поширення помилки крізь час (Backpropagation Through Time, BPTT) градієнт проходить через багато часових кроків, що може призвести до його зникнення (vanishing gradient), чи вибуху (exploding gradient). На противагу цьому LSTM і GRU використовують керовані механізми - ворота (gates), які допомагають зберігати потрібну інформацію протягом довших послідовностей.

LSTM (Long Short-Term Memory) використовує додатковий стан комірки (cell state) і трансформує його за допомогою forget gate, input gate та output gate. Forget gate визначає, яку частину старої пам'яті зберегти, input gate визначає, яку нову інформацію записати, а output gate - яку частину внутрішнього стану передати далі.

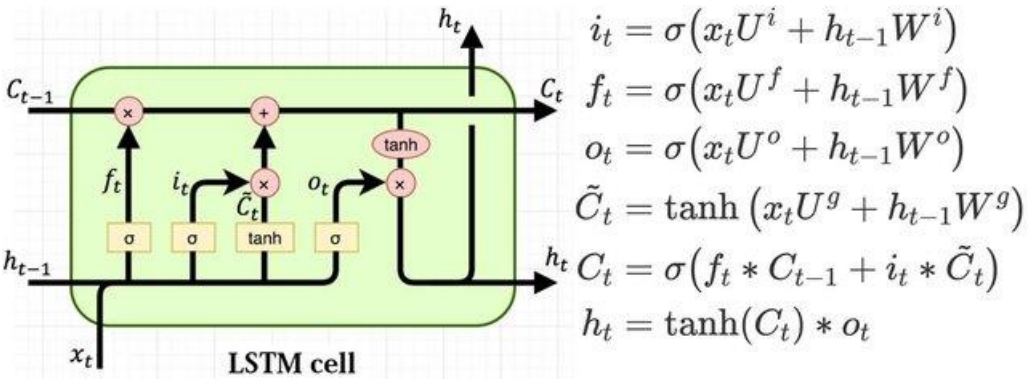


Рис. 9 Архітектура шару LSTM

GRU (Gated Recurrent Unit) є спрощеним варіантом LSTM. Він зазвичай використовує update gate та reset gate і не має окремого стану комірки. Завдяки меншій кількості параметрів GRU може навчатись швидше за LSTM.

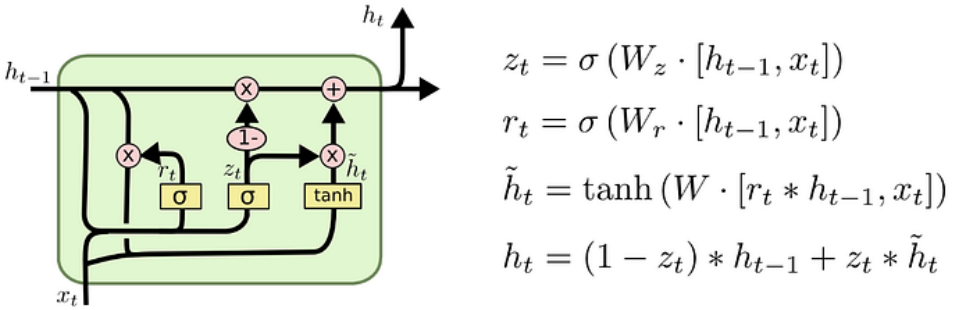


Рис. 10 Архітектура шару GRU

Приклад застосування LSTM/GRU на наборі даних Wall-Following Robot Navigation

Wall-Following Robot Navigation Data містить дані навігації робота SCITOS G5 вздовж стіни, які були виміряні з частотою приблизно 9 разів на секунду.

Варіант sensor_readings_24 містить:

- 24 значення від ультразвукових датчиків;
- Клас, який описує стан руху робота (рух прямо, легкий поворот вправо, різкий поворот вправо, легкий поворот вліво)

Імпортування потрібних бібліотек:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import tensorflow as tf
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
from tensorflow import keras
from tensorflow.keras import layers
```

Забезпечення відтворюваності експериментів:

```
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)
```

Завантаження даних:

```
DATASET_URL = "https://archive.ics.uci.edu/ml/machine-learning-
databases/00194/sensor_readings_24.data"

def load_sensor_dataset(url):
    '''Завантажує набір і додає зрозумілі назви колонок.'''
    column_names = [f"US{i}" for i in range(1, 25)] + ["Class"]
    return pd.read_csv(url, header=None, names=column_names)

df = load_sensor_dataset(DATASET_URL)
```

Кодування класів цілими числами:

```
X_raw = df.drop(columns=["Class"]).to_numpy(dtype=np.float32)
y_raw = df["Class"].astype(str).to_numpy()

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y_raw)
```

Далі виконується поділ набору даних на тренувальну, валідаційну та тестову вибірку. Тут важливим є те, що дані мають послідовну природу і їх не можна довільно перемішувати:

```
def split_ordered_data(features, labels, validation_ratio, test_ratio):
    '''Ділить часові дані без перемішування та зберігає їх порядок.'''
    total_samples = len(features)
    train_end = int(total_samples * (1.0 - validation_ratio - test_ratio))
    validation_end = int(total_samples * (1.0 - test_ratio))

    return (
        (features[:train_end], labels[:train_end]),
        (features[train_end:validation_end], labels[train_end:validation_end]),
```

```
        (features[validation_end:], labels[validation_end:]),
    )

TEST_RATIO = 0.15
VAL_RATIO = 0.15

(X_train_raw, y_train_raw), (X_val_raw, y_val_raw), (X_test_raw, y_test_raw) = (
    split_ordered_data(X_raw, y_encoded, VAL_RATIO, TEST_RATIO)
)
```

Під час нормалізації ознак StandardScaler навчається лише на тренувальній частині - це запобігає витоку інформації з валідаційної та тестової вибірок:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_raw)
X_val_scaled = scaler.transform(X_val_raw)
X_test_scaled = scaler.transform(X_test_raw)
```

Оскільки нейронна мережа при визначенні стану робота має використовувати не тільки поточні виміри сенсорів, а й декілька попередніх потрібно відповідно трансформувати часовий ряд:

```
def create_sequences(features, labels, sequence_length):
    '''Перетворює плоский часовий ряд на часові вікна.'''
    sequences = []
    sequence_labels = []

    for end_index in range(sequence_length - 1, len(features)):
        start_index = end_index - sequence_length + 1
        sequences.append(features[start_index : end_index + 1])
        sequence_labels.append(labels[end_index])

    return (
        np.asarray(sequences, dtype=np.float32),
        np.asarray(sequence_labels, dtype=np.int64),
    )

SEQUENCE_LENGTH = 10

X_train, y_train = create_sequences(
```

```
    X_train_scaled, y_train_raw, SEQUENCE_LENGTH
)
X_val, y_val = create_sequences(X_val_scaled, y_val_raw, SEQUENCE_LENGTH)
X_test, y_test = create_sequences(X_test_scaled, y_test_raw, SEQUENCE_LENGTH)
```

Визначення параметрів тренування:

```
RECURRENT_LAYER = layers.LSTM # або layers.GRU

HIDDEN_SIZE = 64
DROPOUT = 0.2

EPOCHS = 30
BATCH_SIZE = 32
LEARNING_RATE = 0.001
```

Побудова рекурентної моделі:

```
def build_model(recurrent_layer, sequence_length, num_features,
                num_classes, hidden_size, dropout):
    '''Створює класифікатор із вибраним рекурентним шаром.'''
    return keras.Sequential(
        [
            layers.Input(shape=(sequence_length, num_features)),
            recurrent_layer(hidden_size),
            layers.Dropout(dropout),
            layers.Dense(32, activation="relu"),
            layers.Dropout(dropout),
            layers.Dense(num_classes, activation="softmax"),
        ]
    )
NUM_FEATURES = X_train.shape[2]
model = build_model(
    recurrent_layer=RECURRENT_LAYER,
    sequence_length=SEQUENCE_LENGTH,
    num_features=NUM_FEATURES,
    num_classes=NUM_CLASSES,
    hidden_size=HIDDEN_SIZE,
    dropout=DROPOUT,
```

```
)
```

Компіляція моделі:

```
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=LEARNING_RATE),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

Налаштування збереження моделі, дострокової зупинки та керування швидкістю навчання:

```
callbacks = [
    keras.callbacks.ModelCheckpoint(
        "best_robot_model.keras",
        monitor="val_accuracy",
        save_best_only=True
    ),
    keras.callbacks.EarlyStopping(
        monitor="val_loss",
        patience=7,
        restore_best_weights=True
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=3,
        min_lr=1e-6
    )
]
```

Процес тренування:

```
history = model.fit(
    X_train,
    y_train,
    validation_data=(X_val, y_val),
    epochs=30,
    batch_size=32,
```

```
    callbacks=callbacks,
)
```

Оцінювання моделі:

```
from sklearn.metrics import accuracy_score, classification_report

probabilities = best_model.predict(
    X_test,
    verbose=0
)
predictions = np.argmax(
    probabilities,
    axis=1
)
accuracy = accuracy_score(
    y_test,
    predictions
)
print(
    classification_report(
        y_test,
        predictions,
        target_names=CLASS_NAMES
    )
)
```

Корисна література та документація

1. Redmon, J., Divvala, S., Girshick, R., Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection. CVPR, 2016. https://arxiv.org/abs/1506.02640
2. Redmon, J., Farhadi, A. YOLO9000: Better, Faster, Stronger. CVPR, 2017. https://arxiv.org/abs/1612.08242
3. Ultralytics Documentation: YOLO26. https://docs.ultralytics.com/
4. Ultralytics Documentation: Train Mode. https://docs.ultralytics.com/modes/train/
5. Ultralytics Documentation: Predict Mode. https://docs.ultralytics.com/modes/predict/
6. Ultralytics Documentation: TensorBoard Integration. https://docs.ultralytics.com/integrations/tensorboard/

7. Shiri, F. M., Perumal, T., Mustapha, N., Mohamed, R. A Comprehensive Overview and Comparative Analysis on Deep Learning Models: CNN, RNN, LSTM, GRU. Journal on Artificial Intelligence, 6(1), 301–360, 2024. https://arxiv.org/abs/2305.17473

Порядок виконання лабораторної роботи

1. Ознайомитися з теоретичним матеріалом
2. Завантажити набір даних для YOLO та ознайомитись з ним (https://universe.roboflow.com/oms-workspace-t2075/traffic-signs-detection-paz5b/dataset/21). Оскільки набір даних не має тестової вибірки потрібно попередньо перетворити його за допомогою функції `create_stratified_yolo_split`, яку можна знайти нижче.
3. Дотренувати претреновану YOLO26n на завантаженому наборі даних на основі коду з теоретичних матеріалів та проаналізувати:
    - Precision;
    - Recall;
    - mAP50;
    - mAP50–95;
    - mAP для окремих класів;
    - матрицю помилок;
    - приклади передбачень.
4. Виконати індивідуальне завдання, що стосується YOLO.
5. Завантажити набір даних з вимірами сенсорів робота (https://archive.ics.uci.edu/dataset/194/wall+following+robot+navigation+data)
6. Навчити нейронну мережу з LSTM або GRU та проаналізувати:
    - криві навчання;
    - точність;
    - матрицю помилок;
7. Виконати індивідуальне завдання, що стосується LSTM/GRU.
8. Сформувати звіт на основі виконаних завдань

Код для перерозподілу зображень в наборі даних для YOLO:

```
from collections import Counter
from pathlib import Path
from random import Random
from shutil import copy2

def create_stratified_yolo_split(
    source_dir,
    output_dir=None,
    train_ratio=0.8,
    valid_ratio=0.1,
    test_ratio=0.1,
```

```
    seed=42,
):
    """Копіює YOLO-набір у нові приблизно стратифіковані частини."""
    source_dir = Path(source_dir)
    output_dir = Path(output_dir) if output_dir else source_dir.with_name(f"{source_dir.name}_stratified")
    ratios = {"train": train_ratio, "valid": valid_ratio, "test": test_ratio}

    items = []
    for origin in ratios.keys():
        images_dir, labels_dir = source_dir/origin/"images", source_dir/origin/"labels"
        if not images_dir.exists():
            continue
        for image in (path for path in images_dir.iterdir() if path.is_file()):
            label = labels_dir / f"{image.stem}.txt"
            classes = {row[0] for row in read_yolo_labels(label)}
            items.append((image, label, classes, origin))

    totals = Counter(c for _, _, classes, _ in items for c in classes)
    assigned = {s: [] for s in ratios}
    counts = {s: Counter() for s in ratios}
    Random(seed).shuffle(items)
    items.sort(key=lambda item: sum(1 / totals[c] for c in item[2]), reverse=True)

    for item in items:
        classes = item[2]
        split = min(ratios, key=lambda s: (
            sum(counts[s][c] / max(1, totals[c] * ratios[s]) for c in classes)
            + len(assigned[s]) / (len(items) * ratios[s])
        ))
        assigned[split].append(item)
        counts[split].update(classes)

    for split, split_items in assigned.items():
        image_out, label_out = output_dir/split/"images", output_dir/split/"labels"
        image_out.mkdir(parents=True, exist_ok=False)
        label_out.mkdir(parents=True, exist_ok=False)
        for image, label, _, origin in split_items:
            stem = f"{origin}__{image.stem}"
            copy2(image, image_out / f"{stem}{image.suffix}")
            destination = label_out / f"{stem}.txt"
            copy2(label, destination) if label.exists() else destination.touch()

    copy2(source_dir / "data.yaml", output_dir / "data.yaml")
```

Індивідуальні завдання з YOLO

№ варіанта	Тема	Опис
1	Аналіз помилкових детекцій	Знайти false positive і false negative, класифікувати типові причини помилок, показати мінімум 10 прикладів і запропонувати спосіб покращення
2	Порівняння YOLO26n та YOLO26s	Навчити обидві моделі, порівняти mAP, Precision, Recall, розмір моделі, швидкодію. Можна використовувати старішу nano модель замість YOLO26s, якщо немає можливості її навчити.
3	Вплив якості зображення	Створити погіршені набори тестових даних з розмиттям і додаванням шуму, оцінити вплив на метрики.

4	Вплив освітлення	Створити затемнену та пересвічену версію тестового набору даних, оцінити вплив на метрики.
5	Вплив масштабу об’єктів	Розділити об’єкти на малі, середні та великі і визначити для яких модель працює найгірше (за потреби можна модифікувати зображення з набору даних).
6	Вплив дисбалансу класів	Навчити модель на модифікованому наборі даних, де видалено частину екземплярів найрідкіснішого класу та на наборі даних, де за допомогою аугментації додано більше зображень з цим класом. Порівняти оригінальні метрики і метрики для модифікованих наборів даних.
7	Оптимальний поріг впевненості	Знайти значення confidence, що дає найкращий F1-score. Порівняти результати прогнозів з початковим значенням confidence з теоретичних відомостей.
8	Оптимальний поріг впевненості по класам	Для кожного класу знайти confidence, що дає найкращий F1-score. Порівняти результати прогнозів з початковим значенням confidence з теоретичних відомостей
9	Вплив аугментації	Порівняти метрики моделей натренованих без аугментації, з слабкою аугментацією і сильною аугментацією.
10	Вплив різних способів аугментації	Вимикати по одному mosaic, flip, scale, порівняти метрики початкової моделі і моделей натренованих без аугментації.
11	Порівняння претренованої і непретренованої моделі	Порівняти yolo26n.pt із моделлю, створеною з .yaml, дослідити покращення метрик yolo26n.yaml зі збільшенням кількості епох.
12	Навчання на різній кількості даних	Навчити модель на 25%, 50%, 75% від оригінального набору даних, дослідити вплив на метрики.
13	Якість анотацій	Штучно пошкодити частину анотацій: пропустити об’єкти або трохи змістити їх, дослідити вплив на метрики.
14	Розмір зображень	Навчити модель з різними значеннями imgsz (мінімум 3 значення), порівняти швидкодію та метрики.
15	Стабільність до геометричних перетворень	Модифікувати тестовий набір даних за допомогою поворотів, масштабувань та віддзеркалень, оцінити якість прогнозів для кожної модифікації.
16	Вплив положення об’єкта в кадрі	Розділити об’єкти тестового набору на групи за положенням: центр кадру, краї та кути. Порівняти якість прогнозів для різних груп. За потреби створити модифіковані зображення, штучно зміщуючи об’єкти ближче до країв кадру.
17	Вплив заморожування шарів під час донавчання	Навчити модель у кількох режимах: без заморожування шарів, із заморожуванням частини backbone, із заморожуванням більшої кількості шарів. Порівняти час навчання, використання пам’яті та фінальні метрики.
18	Вплив кількості епох та перенавчання	Навчити однакову модель протягом 10, 30, 50 та 100 епох. Проаналізувати метрики, визначити, коли модель перестає суттєво покращуватися та чи виникають ознаки перенавчання.

Індивідуальні завдання з LSTM/GRU

№ варіанта	Тема	Опис
1	Порівняння SimpleRNN, LSTM та GRU	Навчити модель з використанням різних шарів. Порівняти метрики, час навчання та час прогнозування.
2	Вплив довжини часової послідовності	Навчити модель з різними значеннями sequence_length, наприклад 5, 10, 20 і 40. Порівняти метрики та визначити, чи покращується якість зі збільшенням кількості попередніх вимірювань.
3	Кількість сенсорів	Навчити модель на різних файлах з набору даних (2, 4, 24 сенсори). Порівняти якість і складність моделей.
4	Вплив шуму сенсорів	Створити кілька копій тестових даних із доданим випадковим шумом різної інтенсивності. Порівняти метрики з початковими даними та визначити, наскільки модель стійка до похибок сенсорів.
5	Вплив розміру прихованого стану	Навчити однакову LSTM або GRU з різними значеннями hidden_size, наприклад 16, 32, 64 та 128. Порівняти якість, кількість параметрів, час навчання та швидкість прогнозування.
6	Вплив відмови одного сенсора	Вибрати кілька сенсорних каналів і по черзі замінювати їх значення на 0 або середнє значення. Оцінити зміну метрик і визначити, відмова яких сенсорів найбільше впливає на модель.
7	Вплив одночасної відмови кількох сенсорів	Створити варіанти тестового набору, у яких вимкнено 1, 3, 5 та 8 сенсорів. Порівняти метрики та визначити, як зростає похибка зі зменшенням кількості доступних сенсорів.
8	Вплив нормалізації даних	Порівняти навчання без масштабування, зі StandardScaler та з MinMaxScaler. Порівняти швидкість збіжності, функцію втрат та тестові метрики.
9	Вплив способу заповнення пропущених даних	Штучно створити пропущені значення сенсорів та порівняти кілька способів їх заповнення: нулем, попереднім значенням і середнім значенням сенсора. Навчити або протестувати модель та порівняти метрики.
10	Вплив кількості рекурентних шарів	Порівняти модель з одним рекурентним шаром та модель із двома рекурентними шарами. Для багатошарової моделі використати return_sequences=True у першому шарі. Порівняти метрики, кількість параметрів і ознаки перенавчання.
11	Вплив додаткового повнозв'язного шару	Порівняти базову модель з одним Dense-шаром після LSTM/GRU та варіанти з двома або трьома повнозв'язними шарами. Оцінити, чи дає збільшення складності помітне покращення метрик.
12	Аналіз помилкових передбачень	Знайти щонайменше 10 неправильно класифікованих послідовностей, окремо виділити помилки з високою впевненістю. Визначити, які класи плутаються найчастіше.
13	Вплив часової перестановки вимірювань	Для тестових послідовностей створити варіанти, де часові кроки частково або повністю переставлені. Порівняти метрики з оригінальними даними та

		визначити, наскільки модель залежить саме від правильної часової структури.
14	Класифікація за останнім станом проти більш раннього стану	Порівняти два способи формування мітки для одного й того самого часового вікна: клас останнього вимірювання та клас із середини послідовності. Оцінити, як вибір цільового моменту впливає на результати.
15	Введення порога відмови від передбачення	На основі ймовірностей softmax задати кілька порогів упевненості. Якщо максимальна ймовірність нижча за поріг - модель не повертає клас. Для кожного порога оцінити частку відхилених прикладів і точність серед прийнятих передбачень.
16	Вплив згладжування сенсорних даних	Перед формуванням послідовностей застосувати до сенсорних сигналів згладжування, наприклад ковзне середнє для кількох розмірів вікна. Порівняти результати з моделлю, навченою на початкових даних, та визначити, чи допомагає зменшення шуму покращити класифікацію.
17	Вплив похідних сенсорних ознак	Додати до початкових значень сенсорів їхню зміну між сусідніми часовими кроками. Порівняти модель на початкових ознаках і модель з додатковими ознаками зміни. Визначити, чи допомагає інформація про динаміку краще розпізнавати рух робота.
18	Аналіз помилок залежно від швидкості зміни сенсорних даних	Для кожної послідовності обчислити середню абсолютну зміну значень сенсорів між сусідніми часовими кроками. Розділити послідовності на групи з малою, середньою та великою динамікою й порівняти точність для цих груп.

Зміст звіту

Звіт з лабораторної роботи повинен містити:

1. Титульний аркуш із зазначенням назви лабораторної роботи, номера варіанта, прізвища студента та навчальної групи.
2. Номер індивідуального завдання з YOLO, його тему та опис.
3. Номер індивідуального завдання з LSTM/GRU, його текму та опис.
4. Структура набору даних для YOLO та приклади анотованих зображень.
5. Параметри базового експерименту з YOLO: epochs, imgsz, batch, та інші.
6. Графіки функції втрат YOLO.
7. Фінальні метрики YOLO на тестовій вибірці: Precision, Recall, mAP50, mAP50-95, метрики для окремих класів, матриця помилок.
8. Приклади прогнозів YOLO.
9. Результати індивідуального YOLO-завдання з таблицями або графіками.
10. Структура LSTM/GRU-моделі (model.summary()), параметри навчання та використані callbacks.
11. Графіки функції втрат сенсорної моделі
12. Фінальні метрики сенсорної моделі на тестовій вибірці: точність, classification report, матриця помилок.
13. Результати індивідуального LSTM/GRU-завдання.
14. Аналіз результатів та висновки щодо точності моделей та їх ефективності.